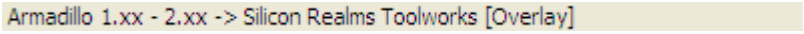
Programa:	Free Registry Fix 3.0
Protección:	Armadillo 2.x + Recupera sólo 50 errores
Objetivos:	Desempakar y evitar la limitación
URL descarga:	http://www.freeregistryfix.com
Dificultad:	Muy sencilla
Herramientas:	OllyDbg v1.10 + PEiD 0.94 + ImpRec 1.6 + LordPE
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 17-11-05

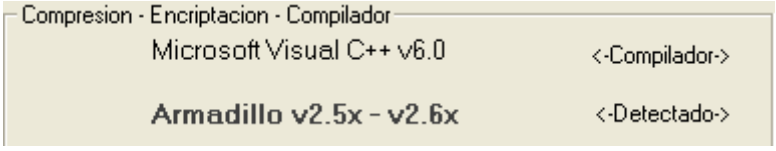
Hola amigos de CracksLatinoS..!!

No me cansaré de saludaros ;-) ni de escribir tutes claro, hace un tiempo me metí en una página web donde descargué mogollón de soft trial entre los que se encuentran este programita para mantener limpio el registro, según dice lo repara, etc, etc., en fin que poco nos interesa ahora lo que haga, de momento vamos a ver qué protección tiene y si somos capaces de superarla, si no lo conseguimos algo habremos aprendido eso fijo.

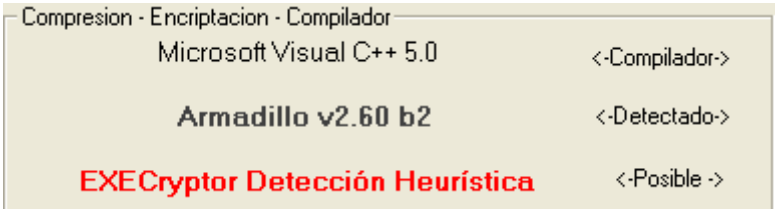
Una vez instalado en la máquina nos abrimos un PEiD donde lo miramos a fondo xDD



Muchas veces nos dice que es esta versión sin serlo, no sé porqué motivo no es capaz de atinar bien con ella, abrimos RDG packer y lo miramos en el modo M-A y dice:



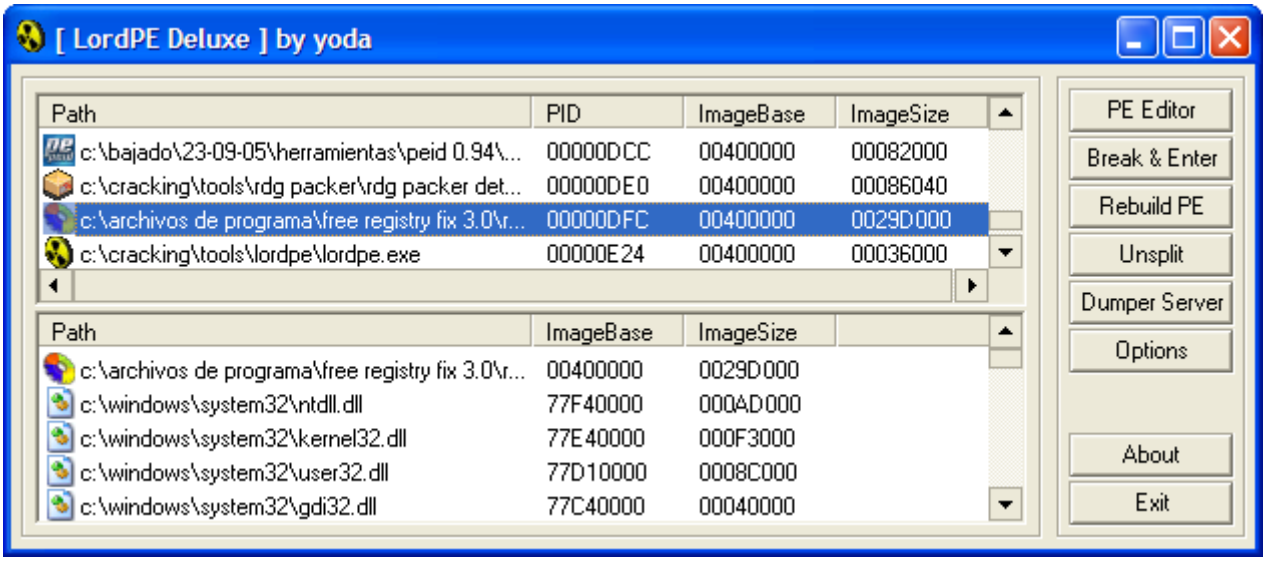
Si lo miramos en el Modo M-B en cambio nos dice esto:



En fin que nadie nos lo dice claro y todavía nos quedan más dudas ahora que antes, jeje… lo que no me gusta nada es que el análisis heurístico detecta EXECryptor ¿? Vaya por Dios ¿qué es esto? El compilador parece ser un Visual C++ esto lo veremos más tarde si es así o no.

Quiero hacer un breve paréntesis en lo que será este tute y lo siguientes, hasta ahora siempre que he sacado un tute a la luz ha sido cuando he desempakado la app y eliminado las limitaciones del Trial, no enviaba el tute en cambio si sólo desempakaba pero no era capaz de evitar las limitaciones, pues esto va a acabar, no veo motivo alguno por el que no enviar el tute del desempakado ¿? Si después hay alguien más hábil capaz de evitar las limitaciones mejor que mejor, el desempakado está ya hecho, entonces este tute verá la luz si consigo el desempakado aunque no le quite las limitaciones, he de decir además que aún no tengo el crack ni nada, lo acabo de instalar, esta es la manera veo yo de explicar el tute con más realismo ya que se explican las cosas tal cual ocurren sin conocer el final.

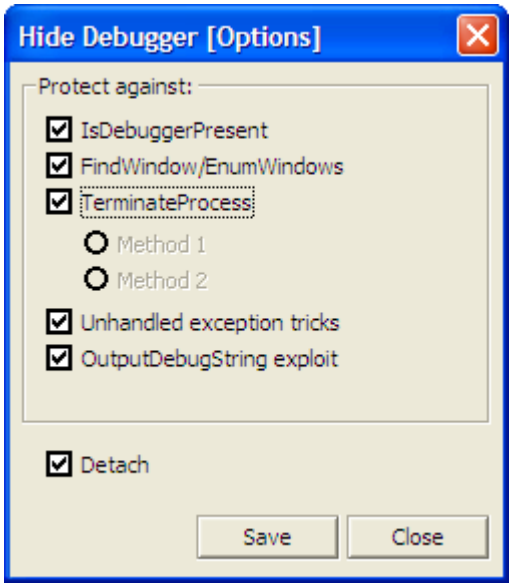
Dicho esto seguimos a lo nuestro, tenemos indicios de que puede tratarse de un Armadillo aunque la versión no la sabemos, para quitarnos las dudas nos valdremos del LordPE, si hay dos procesos en ejecución es un Armadillo con todas las de la ley y sino los hay puede serlo también pero sin CopyMemII (no hay padre), entonces lanzamos el programa y abrimos LordPE, miramos en la lista de procesos y vemos que hay un único proceso corriendo, esto no quiere decir que no sea un Armadillo ni mucho menos, lo que ocurre es que si hubiesen dos sabríamos seguro que lo es (además con Nanomites y/o CopyMem) pero en este caso no es así, pues como digo sólo se carga un único proceso, no hay padre ni hijo en este caso, veámoslo más claro con una imagen del LordPE en la lista de procesos a la que nos refereimos:



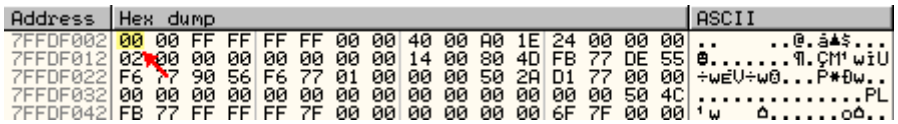
Está claro que hay sólo uno y es el que está resaltado en color azul, entonces cerramos el LordPE, el PEiD y el RDG Packer ya tenemos suficientes datos, abrimos en cambio un Ollydbg 1.10 con los pluggins:

- IsDebuggerPresent 1.3
- CommandBar 3.00.108
- HideDebugger 1.2.3f by Asterix
- OllyDump 2.21.108 by Gigapede
- OllyScript 0.92 by SHaG

comenzamos con estos pluggins de momento, el HideDebugger lo configuramos al máximo con todas las opciones activadas para que no nos detecten Olly:



Tenemos activada la opción IsDebuggerPresent que nos pone el byte de debugger a 0, recordad que en el EP estará en [EBX+2] y en XP SP1 estará en [7FFDF002] cuyo byte bajo debe estar a cero:



Como vemos estando en el EP de este programa lo miramos y está efectivamente a 0, si no tuviésemos los plugins que ocultan el debugger estaría a 1 y sería fácilmente detectado. La opción FindWindow/EnumWindows la activamos también para evitar que nos pillen por el Caption de la ventana o el WindowClass, las llamadas a FindWindowA y EnumWindows pueden ser peligrosas, recordemos como Acprotect nos detecta de esta manera además de muchas otras, el TerminateProcess que no sé para que diablos sirve pero lo activamos por si las moscas ¿? El "Unhandled exception tricks" lo activamos también, esto es para salvar

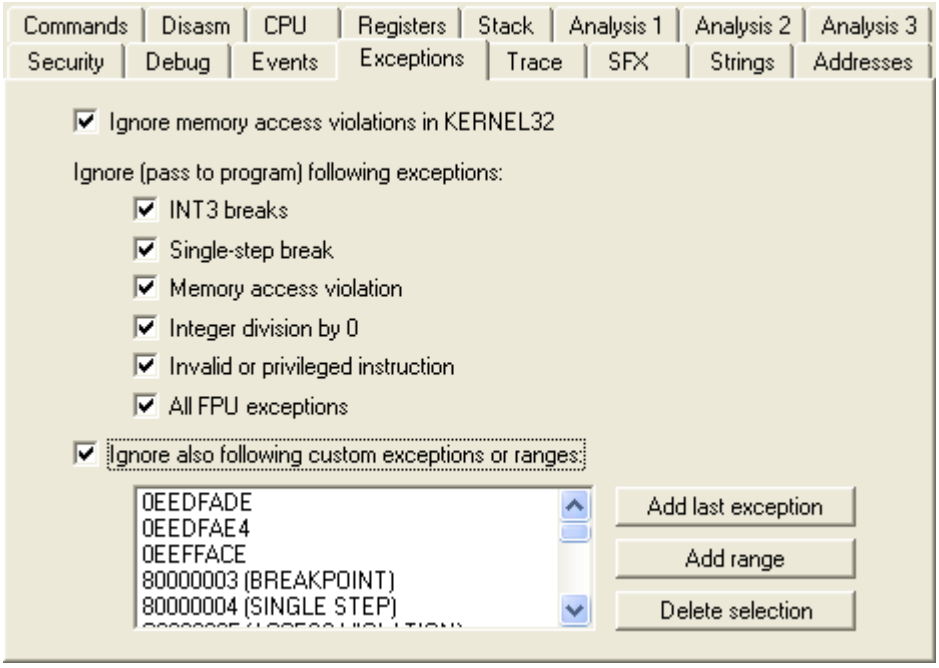
las excepciones que se produzcan, para una mayor rpofundización en el tema recomiendo leer los siguientes tutes y teorías:

17-ANTIDEBUGGING 1.zip
18-ANTIDEBUGGING 2.zip
19-ANTIDEBUGGING 3.zip
20-ANTIDEBUGGING 4.zip
21-ANTIDEBUGGING 5.zip
22-ANTIDEBUGGING 6.zip
373-Mtodos_Antidebugging_7_JuanJose.rar
526-EPROCESS_TEB_PEB_TLS_RegistroFS (antidebugging).rar
538-EPROCESS_TEB_PEB_TLS_RegistroFS por kaos Xlro.rar
539-EPROCESS_TEB_PEB_TLS_RegistroFS_II por kaos Xlro.rar

Creo que la 526 y la 539 son las mismas ¿? Bueno lo mismo va a dar, estos son teorías sobre antidebugging, de verdad que si alguien quiere llegar lejos en esto del cracking debe primero estudiar estos conceptos, son fundamentales para avanzar y más teniendo en cuenta que cada día se utilizán más técnicas de este tipo para evitar lso debuggers, sino mirad un EXECryptor a ver cuanto tiempo os lleva llegar al OEP, jeje.. y todo gracias a estas estructuras, ya os digo un tesoro estos tutes, mi enhorabuena a los autores xDDD

Siguiendo con la configuración del pluggin HideDebugger nos queda solamente la activación de la casilla "OuputDebugString exploit" este truco lo utiliza bastante el Armadillo en sus últimas versiones, se trata de enviarle una cadena al debugger que no puede procesar y casque, normalmente termina con el cierre de Olly (se cierra de mala manera, jeje) y con esto terminamos de configurar el pluggin, para los demás no hay problema los ponemos en la misma carpeta del Olly y listos no hay que configurar nada.

Después tildamos todas las excepciones para que Olly no maneje ninguna:



Ahora ya estamos preparados para enfrentarnos a este Armadillo, reiniciamos Olly (la configuración que hemos heho lo requiere) y nos detenemos en el EP del packer:

```
0050BC4D  $  55          PUSH EBP                          ;ENTRY POINT!!
0050BC4E  .  8BEC        MOV EBP,ESP
0050BC50  .  6A FF       PUSH -1
0050BC52  .  68 307A5200 PUSH regfix.00527A30
0050BC57  .  68 30B65000 PUSH regfix.0050B630              SE handler installation
0050BC5C  .  64:A1 000000( MOV EAX,DWORD PTR FS:[0]
0050BC62  .  50          PUSH EAX
0050BC63  .  64:8925 0000( MOV DWORD PTR FS:[0],ESP
0050BC6A  .  83EC 58     SUB ESP,58
```

Ese "PUSH EBP" es el EntryPont (primera ionstrucción a ejecutar) que no es el OEP claro, éste lo deberemos descubrir después, de momento veamos si corre en Olly pulsamos F9 y corre perfecto, parece que los pluggins han hecho bien su trabajo ;-)

Reiniciamos Olly y quedamos en el EP, desde aquí ponemos ya un BPM en la primera sección que es la que viene detrás del PE header:

```
Address  Size     Owner   Section  Contains     Type Access  Initial Mapp
00010000 00001000                               Priv RW      RW
00020000 00001000                               Priv RW      RW
0012B000 00001000                               Priv RW  Gua RW
0012C000 00004000                stack of ma    Priv RW  Gua RW
00130000 00001000                               Map  R       R
00140000 00002000                               Map  R       R
00150000 00004000                               Priv RW      RW
00250000 00006000                               Priv RW      RW
00260000 00001000                               Map  RW      RW
00270000 00016000                               Map  R       R       \Dev
00290000 00034000                               Map  R       R       \Dev
002D0000 00041000                               Map  R       R       \Dev
00320000 00006000                               Map  R       R       \Dev
00330000 00041000                               Map  R       R
00380000 00001000                               Priv RWE     RWE
00390000 00001000                               Priv RWE     RWE
003A0000 00001000                               Priv RWE     RWE
003B0000 00008000                               Priv RW      RW
003C0000 00001000                               Priv RW      RW
003D0000 00001000                               Priv RW      RW
00400000 00001000 regfix         PE header      Imag R       RWE
00401000 000C9000 regfix CODE
004CA000 00003000 regfix DATA
004CD000 00003000 regfix BSS
004D0000 00003000 regfix .idata
004D3000 00001000 regfix .tls
004D4000 00001000 regfix .rdata
004D5000 0000D000 regfix .reloc
004E2000 00030000 regfix .text
00512000 00010000 regfix .adata
00522000 00010000 regfix .data
00532000 00010000 regfix .reloc
00542000 000D0000 regfix .pdata
00612000 0008B000 regfix .rsrc
006A0000 0000B000
00760000 00002000
```

```
Actualize
Dump in CPU
Dump
Search                          Ctrl+B
Set break-on-access             F2
Set memory breakpoint on access
Set memory breakpoint on write
```

Y damos RUN para que corra el programa, la próxima parada es directamente en el OEP del programa protegido, esta vez ha sido fácil ;-)

```
004C8FF8  55            PUSH EBP                          ;El OEP!!
004C8FF9  8BEC          MOV EBP,ESP
004C8FFB  83C4 F0       ADD ESP,-10
004C8FFE  53            PUSH EBX
004C8FFF  B8 588C4C00   MOV EAX,regfix.004C8C58
004C9004  E8 5BD8F3FF   CALL regfix.00406864
004C9009  68 E4904C00   PUSH regfix.004C90E4              ASCII "FreeRegistryFix.Demo"
004C900E  6A FF         PUSH -1
004C9010  6A 00         PUSH 0
004C9012  E8 59DBF3FF   CALL regfix.00406B70
004C9017  A3 ECF64C00   MOV DWORD PTR DS:[4CF6EC],EAX
```

Lo anotamos como siempre hacemos:

OEP: 4C8FF8

Ya que estamos vamos a por la IAT, como parece que es un .exe compilado en Delphi (por la estructura del código y las secciones que vemos que tiene CODE, DATA, BSS…) sabemos que el primer CALL debe ser a GetModuleHandleA, lo seguimos pulsando Enter y caemos en:

```
00406864  53            PUSH EBX
00406865  8BD8          MOV EBX,EAX
00406867  33C0          XOR EAX,EAX
00406869  A3 9CA04C00   MOV DWORD PTR DS:[4CA09C],EAX
0040686E  6A 00         PUSH 0
00406870  E8 2BFFFFFF   CALL regfix.004067A0              => ENTRAREMOS!!
00406875  A3 68D64C00   MOV DWORD PTR DS:[4CD668],EAX
```

Aún no vemos llamadas a API's entramos en el CALL de :406870 igual que antes, o sea pulsando la tecla Enter una vez situados sobre él y caemos aquí ;-)

```
004067A0 - FF25 A0024D00 JMP DWORD PTR DS:[4D02A0]
004067A6   8BC0          MOV EAX,EAX
004067A8 - FF25 9C024D00 JMP DWORD PTR DS:[4D029C]        kernel32.LocalAlloc
004067AE   8BC0          MOV EAX,EAX
004067B0 - FF25 98024D00 JMP DWORD PTR DS:[4D0298]        kernel32.TlsGetValue
004067B6   8BC0          MOV EAX,EAX
004067B8 - FF25 94024D00 JMP DWORD PTR DS:[4D0294]        kernel32.TlsSetValue
004067BE   8BC0          MOV EAX,EAX
```

Unos cuantos saltos a la IAT, jeje.. el primero es al que apunta el CALL y sabemos que va a GetModuleHandleA aunque no lo ponga al lado como en las demás. Antes de que se me olvide ponemos un HBP en el OEP :4C8FF8 para cuando reiniciemos Olly no tener que andar poniendo el BPM en la primera sección ni nada, así paramos en el OEP directamente sin problemas ;-)

En cuanto a la IAT la delimitamos, nos situamos sobre el JMP de la imagen anterior y picamos con el botón derecho, elejimos "Follow in dump->Memory address" para caer en plena tabla, ahora se trata de delimitarla buscando el inicio y el final, desde donde caemos vamos subiendo un poco con los cursores o con la rueda del mouse hasta encontrar:

```
Address   Hex dump                                          ASCII
004D01AC  00 00 00 00 7A 2C 0D 00 00 00 00 00 00 00 00 00  ....z,..........
004D01BC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004D01CC  CA 25 F4 77 90 56 F6 77 DE 55 F6 77 45 A7 E5 77
004D01DC  CB 15 E6 77 72 AC E5 77 A0 60 E5 77 82 A6 E5 77
004D01EC  CD E6 C8 00 DF A7 E5 77 71 A6 E5 77 60 A6 E5 77
004D01FC  44 7F E5 77 49 A9 E5 77 FC A7 E5 77 E1 60 E5 77
```

Y lo anotamos una vez lo tenemos:

iat_inicio: 4D01CC

Hacemos lo mismo con el final:

```
Address   Hex dump                                          ASCII
004D0998  6C AA C8 00 FF E9 C8 00 05 AA C8 00 0E DF DA 77
004D09A8  4B 22 DA 77 4F 02 DB 77 90 6B DA 77 1E 21 DA 77
004D09B8  C0 AA C8 00 33 84 B1 76 18 AA C8 00 62 8B B0 76
004D09C8  C0 AA C8 00 6B 65 72 6E 65 6C 33 32 2E 64 6C 6C  ....kernel32.dll
004D09D8  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004D09E8  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

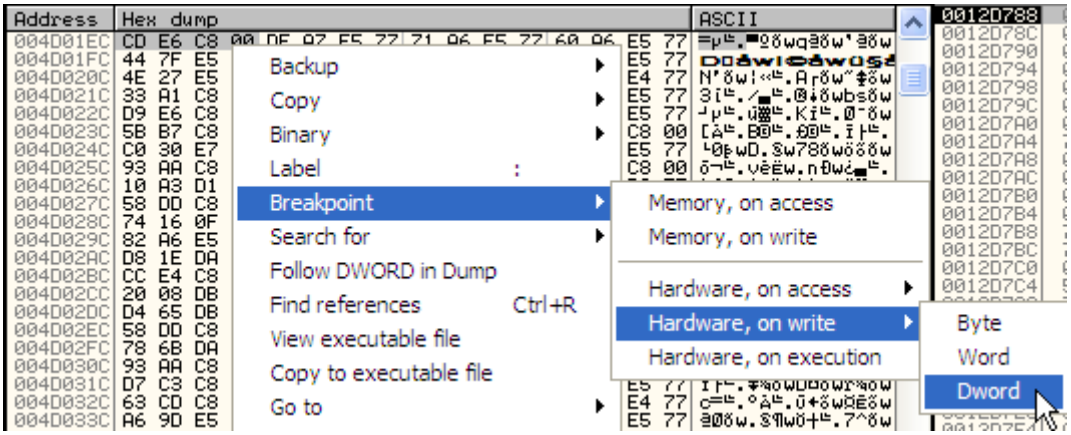
ya lo tenemos también, igualmente lo anotamos:

iat_final: 4D09C8

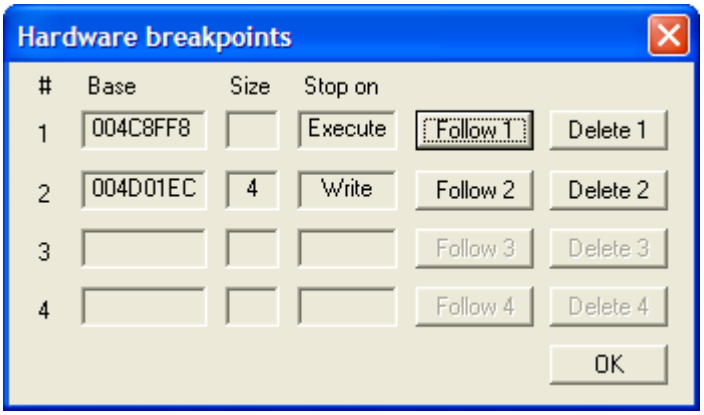
Restando ambos valores obtenemos el largo:

iat_largo: 7FC

Una vez tenemos la tabla totalmente delimitada por ambos lados y con du longitud y todo procedemos a buscar el mágico, el modo de hacerlo es poniendo un HBPWD en un valor MALO de la mismo, yo suelo elegir el primero que encuentro, en este caso [4D01EC]==C8E6CD, nos anotamos los bytes finales C8E6CD que nos servirán de guía y le ponemos el HBP:



Ahora reiniciamos Olly nuevamente, a estas alturas deberemos tener dos HBP's puestos: el del OEP (en ejecución) y el de la entrada MALA (por escritura):



Con estos dos puestos de momento nos vale, como dije antes reiniciamos Olly y pulsamos F9 para dejar correr la aplicación, comenzará a saltar debido al HBP de escritura en la entrada MALA, la primera vez que lo hace es aquí:

```
77C12F43   F3:A5          REP MOVS DWORD PTR ES:[EDI],DWORD PTR D
77C12F45   FF2495 5830C17 JMP DWORD PTR DS:[EDX*4+77C13058]
77C12F4C   8BC7           MOV EAX,EDI
77C12F4E   BA 03000000    MOV EDX,3
77C12F53   83E9 04        SUB ECX,4
77C12F56 v 72 0C          JB SHORT msvcrt.77C12F64
```

Pero no escribe los bytes finales todavía, entonces no nos sirve volvemos a pulsar F9 y para en este otra parte:

```
00CA592B v 73 1D          JNB SHORT 00CA594A
00CA592D   8B85 10D8FFFF  MOV EAX,DWORD PTR SS:[EBP-27F0]
00CA5933   8B8D 64D2FFFF  MOV ECX,DWORD PTR SS:[EBP-2D9C]
00CA5939   8908           MOV DWORD PTR DS:[EAX],ECX          ;AQUI GUARDA!!
00CA593B   8B85 10D8FFFF  MOV EAX,DWORD PTR SS:[EBP-27F0]     regfix.004D01EC
00CA5941   83C0 04        ADD EAX,4
00CA5944   8985 10D8FFFF  MOV DWORD PTR SS:[EBP-27F0],EAX
00CA594A ^ E9 CEFCFFFF    JMP 00CA561D
```

Esta sí que nos interesa pues guarda los bytes que buscamos, además si nos fijamos vemos que estamos en un bucle que termina en :CA594A y empieza en :CA561D entre estas dos direcciones va rellenando la tabla, a partir de aquí es cuestión de traceo en busca del mágico, iremos con F8 de momento a ver donde pone la API correcta en EAX, cuando lo encontremos veremos porqué no guarda este valor en la tabla y tendremos el mágico rápidamente localizado:

```
00CA57EB v 75 42          JNZ SHORT 00CA582F                  ;SALTO MAGICO!!
00CA57ED   0FB785 68D2FFF MOVZX EAX,WORD PTR SS:[EBP-2D98]
00CA57F4   85C0           TEST EAX,EAX
00CA57F6 v 74 0F          JE SHORT 00CA5807
00CA57F8   0FB785 68D2FFF MOVZX EAX,WORD PTR SS:[EBP-2D98]
00CA57FF   8985 84BDFFFF  MOV DWORD PTR SS:[EBP+FFFFBD84],EAX
00CA5805 v EB 0C          JMP SHORT 00CA5813
00CA5807   8B85 60D2FFFF  MOV EAX,DWORD PTR SS:[EBP-2DA0]
00CA580D   8985 84BDFFFF  MOV DWORD PTR SS:[EBP+FFFFBD84],EAX
00CA5813   6A 01          PUSH 1
00CA5815   FFB5 84BDFFFF  PUSH DWORD PTR SS:[EBP+FFFFBD84]
00CA581B   FFB5 9CD4FFFF  PUSH DWORD PTR SS:[EBP-2B64]
00CA5821   E8 C346FEFF    CALL 00C89EE9                       ;pone la API en EAX!!
00CA5826   83C4 0C        ADD ESP,0C
00CA5829   8985 64D2FFFF  MOV DWORD PTR SS:[EBP-2D9C],EAX
00CA582F   83BD 64D2FFFF  CMP DWORD PTR SS:[EBP-2D9C],0
00CA5836 v 75 42          JNZ SHORT 00CA587A
00CA5838   0FB785 68D2FFF MOVZX EAX,WORD PTR SS:[EBP-2D98]
```

El mágico está en :CA57EB como podemos comprobar, si ponemos un BP en el salto y otro donde guarda lo comprobaremos:

```
00CA57EB v 75 42          JNZ SHORT 00CA582F                  ;NO SALTA..!!
00CA57ED   0FB785 68D2FFF MOVZX EAX,WORD PTR SS:[EBP-2D98]
00CA57F4   85C0           TEST EAX,EAX
00CA57F6 v 74 0F          JE SHORT 00CA5807
```

Veamos lo que guarda (no va a saltar):

```
00CA592D   8B85 10D8FFFF  MOV EAX,DWORD PTR SS:[EBP-27F0]
00CA5933   8B8D 64D2FFFF  MOV ECX,DWORD PTR SS:[EBP-2D9C]
00CA5939   8908           MOV DWORD PTR DS:[EAX],ECX          kernel32.GetStartupInfoA
00CA593B   8B85 10D8FFFF  MOV EAX,DWORD PTR SS:[EBP-27F0]
```

Guarda una API válida, seguimos pulsando F9 hasta que veamos que va a saltar:

```
00CA57EB v 75 42          JNZ SHORT 00CA582F                  ;SI SALTA..!!
00CA57ED   0FB785 68D2FFF MOVZX EAX,WORD PTR SS:[EBP-2D98]
00CA57F4   85C0           TEST EAX,EAX
00CA57F6 v 74 0F          JE SHORT 00CA5807
```

Veamos ahora lo que guarda cuando sí va a saltar:

```
00CA592D   8B85 10D8FFFF  MOV EAX,DWORD PTR SS:[EBP-27F0]
00CA5933   8B8D 64D2FFFF  MOV ECX,DWORD PTR SS:[EBP-2D9C]
00CA5939   8908           MOV DWORD PTR DS:[EAX],ECX
00CA593B   8B85 10D8FFFF  MOV EAX,DWORD PTR SS:[EBP-27F0]
```

Guarda un valor MALO, si nos fijamos ahora no vemos el nombre de ninguna API al lado como antes, y si miramos los registros veremos el valor de ECX que es el que guarda:

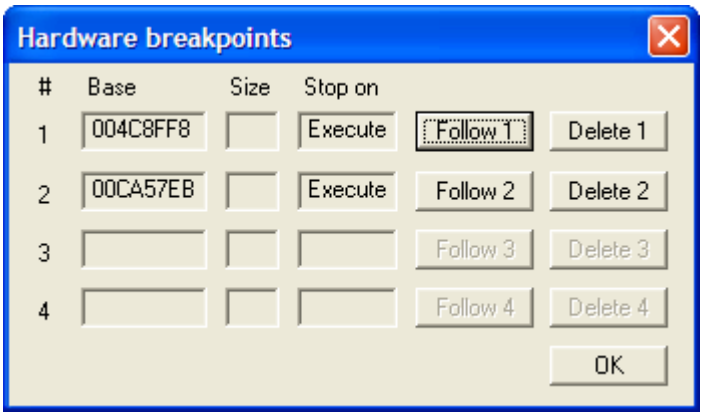
```
Registers (FPU)
EAX 004D021C regfix.004D021C
ECX 00C8A133
EDX 0012A0C8 ASCII "GetProcAddress"
EBX 2A900012
ESP 00128CE4
```

Ahí está, vemos que ECX==C8A133 este valor es el que meterá en la IAT en lugar de la dirección de la API correcta, esto es así porque saltó en :CA57EB y no pasó por el call GetProcAddress donde pone la API en EAX que está en :CA5821…

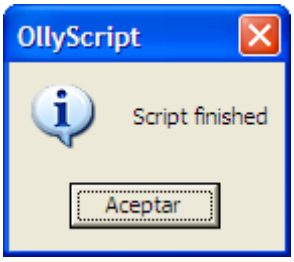
Para solucionar esto nos valdremos de un sencillo script que lo único que hará será simular el nopeo del salto mágico, es este:

```
-----------/ inicio_script
inicio:
eob vamos
run

vamos:
cmp eip, 0ca57eb
jne termina
mov eip, 0ca57ed
jmp inicio

termina:
ret
-----------/ final_script
```

Lo que hace es mirar si se produce o no una excepción o BPX, si es así pasa el control a la etiqueta "vamos" donde mira en qué dirección sucedió, si es en el mágico cambiamos el valor del EIP simulando el nopeo del mismo, y sino es allí significa que estamos en el OEP y el script termina por eso vamos a la etiqueta "termina" donde retornamos el control a Olly y estaremos detenidos en el OEP. Para que el script funcione deberemos tener un HBP en la dirección del mágico y otro en el OEP para parar allí una vez termine éste, con estos dos nos bastan:



Entonces reiniciamos Olly, nos aseguramos de tener todas las excepciones desactivadas (tildando las casillas) como mostramos al inicio, el script lo podemos lanzar tranquilamente desde el EntryPoint siempre y cuando no tengamos BP's puestos ni HBP's ni BPM's ni nada de nada, sólo los dos que vimos antes, cuando termine:



Nos aparece este menajito, jeje.. que buena pinta tiene esto, estamos además en el OEP gracias al segundo HBP que teníamos puesto por eso dije que siempre es bueno tener uno en el OEP, mirad si era cierto, ahora seguimos el primer call como antes hicimos a ver a qué API llama y vemos esto:

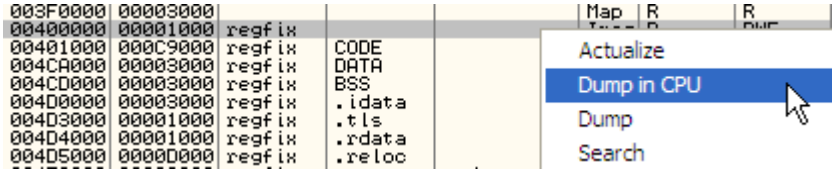
```
004067A0 - FF25 A0024D00  JMP DWORD PTR DS:[4D02A0]   kernel32.GetModuleHandleA
004067A6   8BC0           MOV EAX,EAX
004067A8 - FF25 9C024D00  JMP DWORD PTR DS:[4D029C]   kernel32.LocalAlloc
004067AE   8BC0           MOV EAX,EAX
004067B0 - FF25 98024D00  JMP DWORD PTR DS:[4D0298]   kernel32.TlsGetValue
004067B6   8BC0           MOV EAX,EAX
004067B8 - FF25 94024D00  JMP DWORD PTR DS:[4D0294]   kernel32.TlsSetValue
004067BE   8BC0           MOV EAX,EAX
```

Ahora parece estar mejor pues aparece "GetModuleleHandleA" al lado del JMP cosa que antes no ocurría, esto indica que la IAT está correcta ahora, vamos a dumpear xDD pero antes sabemos que Armadillo nos hace un par de trastadas en el header, en concreto:

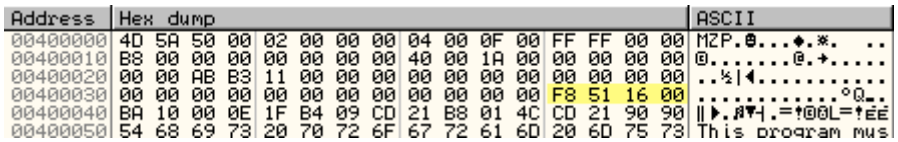
- Nos cambia la “PE Signature”
- Nos cambia el número de secciones

El tema de la Signature es este: en el offset 0x3Ch tenemos un valor, este valor debe ser el offset donde esté la cadena “PE” es decir, en la dirección :40003C (suponemos que tenemos un imagebase de 400000 en el módulo) tenemos el valor 0x100h lo que quiere decir que en la dirección 100+400000=400100 debe estar la cadena “PE” ok?

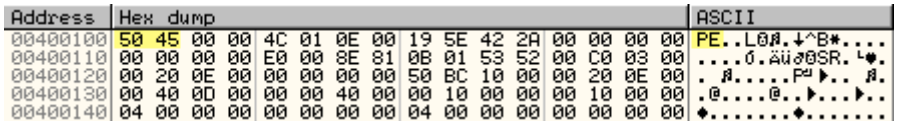
Veámoslo más claro con un ejemplo, repararemos de paso la de este archivo y matamos dos pájaros de un tiro, primero de nada vamos al Memory Map y volcamos la sección de la cabecera que es la primera del archivo que comienza en la dirección del imagebase, es decir si tenemos imagebase==400000 el header comienza precisamente en :400000 entonces vamos a esta dirección y volcamos la sección en la ventana de dump:



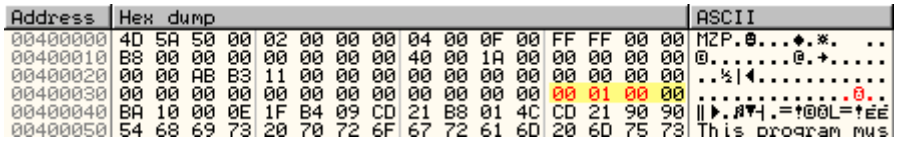
Esto nos pondrá los datos de la cabecera en la ventana de datos (ventana dump) allí vamos al offset 0x3Ch (dirección :40003C) y vemos este valor:



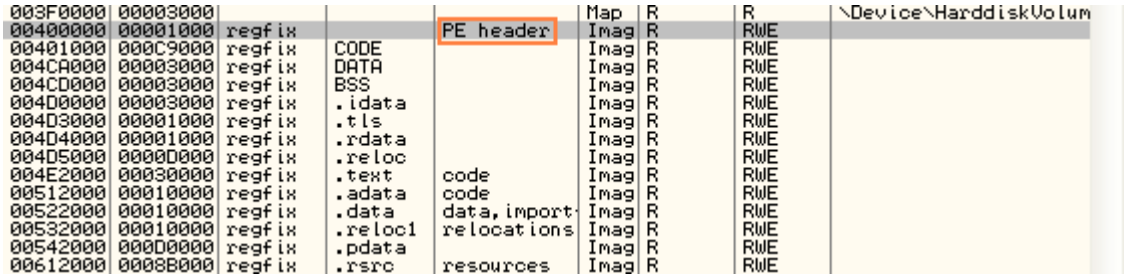
Encontramos el valor 1651F8, vale ahora busquemos la direcicón donde está la cadena “PE” de la que hablaba antes, está aquí:



La encontramos en la dirección :400100 cuyo offset es 400100-400000=100h este valor 0x100h es el que debe aparecer en lugar del 1651F8 en la Signature así que lo cambiamos:



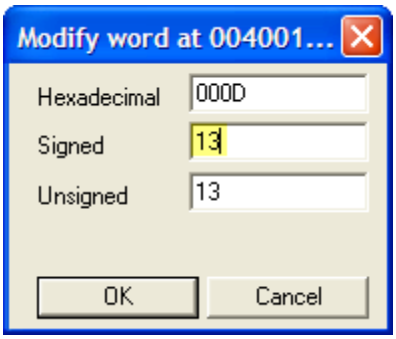
Y ya tenemos reparado el primer tema, si ahora vamos a la ventana de memoria (Memory Map) vemos que la sección ya vuelve a tener el indicador “PE header” que no tenía:



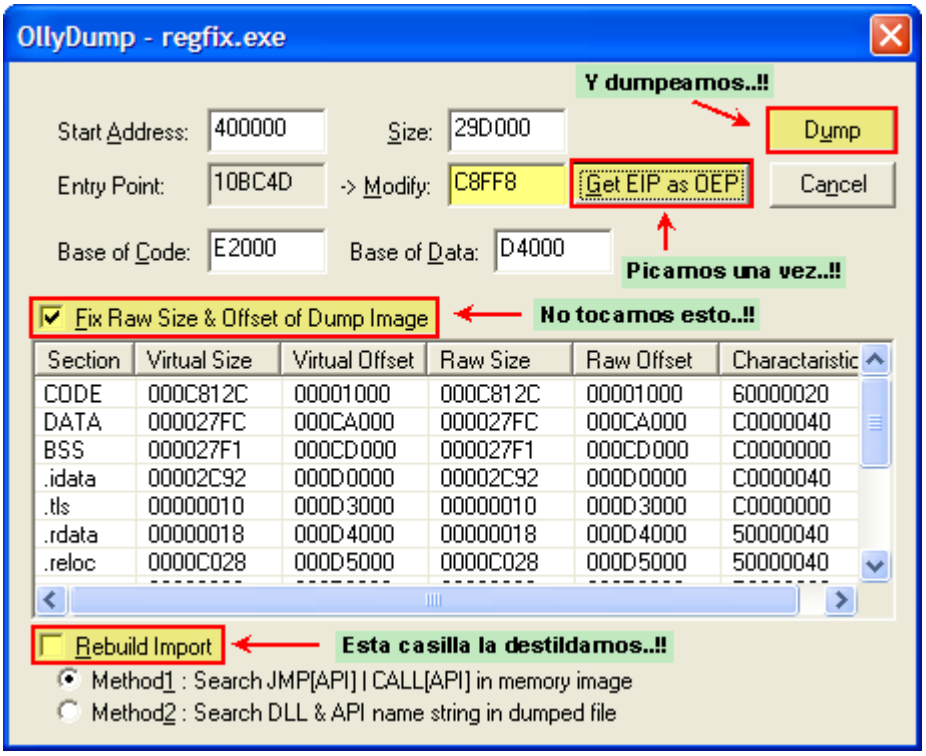
Esto quiere decir que ya tenemos la mitad del trabajo hecho, vamos a por la otra mitad: el número de secciones, para repararlo es bien fácil, abrimos el .exe en Peditor o cualquier otro editor de cabeceras y miramos el número de secciones del archivo, esto también se puede hacer con Olly pero esto ya lo expliqué en otros tutes de Armadillo, vemos que en realidad tiene 13 secciones, ahora vamos a ver cuantas secciones nos ha puesto Armadillo para este archivo, esto se hace yendo a la ventana de memoria, seleccionando la sección de la cabecera y haciendo doble clic sobre ella, entonces se abrirá una ventanita donde aparecerá toda la información de la misma, aquí buscamos el número de secciones que dice tener el archivo:

```
Dump - regfix 00400000..00400FFF
00400100   50 45 00 0(ASCII "PE"      PE signature (PE)
00400104   4C01       DW 014C         Machine = IMAGE_FILE_MACHINE_I386
00400106   0E00       DW 000E          NumberOfSections = E (14.)
00400108   195E422A   DD 2A425E19      TimeDateStamp = 2A425E19
0040010C   00000000   DD 00000000      PointerToSymbolTable = 0
00400110   00000000   DD 00000000      NumberOfSymbols = 0
00400114   E000       DW 00E0          SizeOfOptionalHeader = E0 (224.)
00400116   8E81       DW 818E          Characteristics = EXECUTABLE_IMAGE!32BIT_MACHINE
00400118   0B01       DW 010B         MagicNumber = PE32
0040011A   53         DB 53            MajorLinkerVersion = 53 (83.)
0040011B   52         DB 52            MinorLinkerVersion = 52 (82.)
0040011C   00C00300   DD 0003C000      SizeOfCode = 3C000 (245760.)
00400120   00200E00   DD 000E2000      SizeOfInitializedData = E2000 (925696.)
00400124   00000000   DD 00000000      SizeOfUninitializedData = 0
00400128   50BC1000   DD 0010BC50      AddressOfEntryPoint = 10BC50
0040012C   00200E00   DD 000E2000      BaseOfCode = E2000
```

Como vemos nos dice que tiene 14 cuando en realidad sólo tiene 13, modificamos el número por el correcto, picamos con el botón derecho sobre este valor y elejimos la opción "Modify integer" que aparece, se nos abrirá un cuadrito donde lo cambiamos:

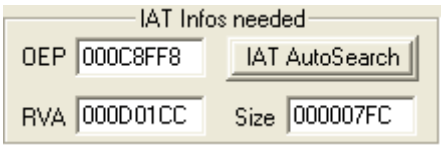


Ya está cambiado, aceptamos y listo podemos dumpear con las opciones de siempre ;-)

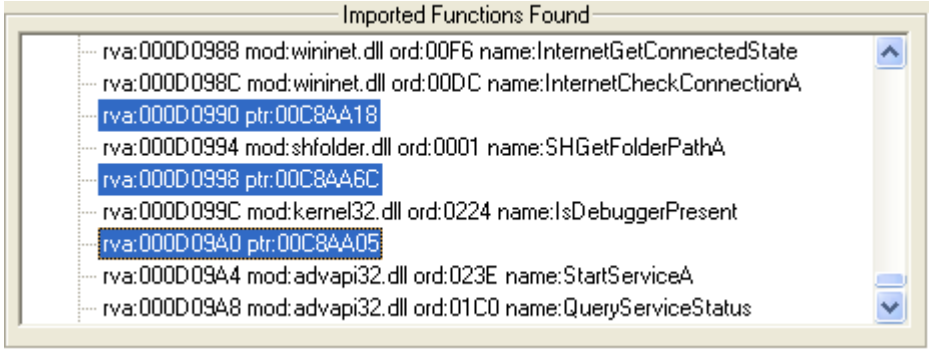


Salvamos el archivo como tute.exe, ahora sólo nos queda reparar la IAT, que en realidad ni eso ya que el mágico nos ha hecho el trabajo ;-)

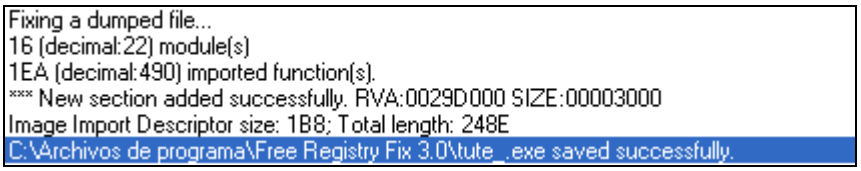
Como ya tenemos el dumpeado y la IAT reparada sólo nos queda abrir el Import Reconstructor 1.6 y darle los datos correctos:



Picamos en "Get Imports" y nos dice que quedan 21 entradas por resolver pero si las miramos detenidamente nos damos cuenta que no es así, sino que son la separación entre distintas DLL's como vemos en la imagen:



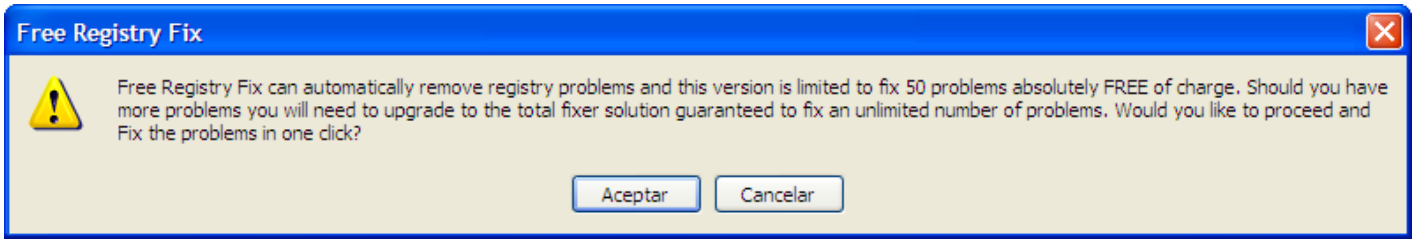
Le hacemos un "Cut Thunks" a todas (a las 21) y nos dice que no quedan más por reparar ;-) hacemos un "Fix Dump" y elejimos tute.exe:



Y nos informa que el archivo tute_.exe se salvó satisfactoriamente, lo probamos y…

jejeje… otro Armadillo más al bolsillo, esto rula pero bien xDD

Ahora que tenemos el desempakado vamos a tratar de eliminar las limitaciones del programa, hasta ahora sólo vi una y es que sólo reapara 50 entradas del registro, para llegar a ella hacemos un Scan y cuando termine le decimos que repare los errores que encontró que fueron 194 en mi máquina, vaya si encontró no? Bueno le digo que los repare y me encuentro con esta desagradable sorpresa:



Cancelo para que no repare nada y se lo vuelvo a decir ;-) pero antes le preparamos una sorpresita a él desde el CommandBar de Olly:

Una vez puesto entonces sí le decimos que los repare, antes de que aparezca el mensaje en pantalla Olly se detiene, miremos el stack (en la pila) y veremos esto:

```
0012FBE8  004B4251  CALL to MessageBoxA from tute_.004B424C
0012FBEC  00120886  hOwner = 00120886 ('Free Registry Fix',class='FREEREGFIX.CLASS',parent=000807AC)
0012FBF0  004B44C8  Text = "Free Registry Fix can automatically remove registry problems and this version
0012FBF4  00D898B8  Title = "Free Registry Fix"
0012FBF8  00000031  Style = MB_OKCANCEL|MB_ICONEXCLAMATION|MB_APPLMODAL
0012FBFC  0012FDEC  Pointer to next SEH record
0012FC00  004B44B4  SE handler
```

Ahí están los parámetros de la llamada a la API, lo que nos interesa está en la primera línea donde dice "CALL to … from tute_004B424C" esta dirección :4B424C es la que nos interesa pues desde ella se realiza la llamada al mensaje, vamos para allí y vemos esto:

```
004B424C   E8 3732F5FF   CALL <JMP.&user32.MessageBoxA>   ;Llama al mensaje..!!
004B4251   83F8 02       CMP EAX,2
004B4254 v 75 13         JNZ SHORT tute_.004B4269
004B4256   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
```

Efectivamente la llamada está ahí en :4B424C como la pila nos decía, detrás lleva una CMP entre EAX y 2, esto es para saber qué botón pulsamos si "OK" o "Cancel" si es Cancel salimos con EAX==2 y no salta, entonces no hace la reparación, nos llama la atención un salto que hay poco más arriba del mensaje:

```
004B41D5   E8 96710000   CALL tute_.004BB370
004B41DA   85C0          TEST EAX,EAX
004B41DC v 0F84 AF020000 JE tute_.004B4491                ;ojo a este salto ¬?
004B41E2   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
004B41E5   C680 24050000 MOV BYTE PTR DS:[EAX+524],1
004B41EC   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
004B41EF   8B80 0C040000 MOV EAX,DWORD PTR DS:[EAX+40C]
004B41F5   33D2          XOR EDX,EDX
004B41F7   E8 74F1F7FF   CALL tute_.00433370
004B41FC   E8 476AF5FF   CALL tute_.0040AC48
004B4201   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
004B4204   DC98 B0040000 FCOMP QWORD PTR DS:[EAX+4B0]
004B420A   DFE0          FSTSW AX
004B420C   9E            SAHF
004B420D v 0F82 2C020000 JB tute_.004B443F
004B4213   8D45 F8       LEA EAX,DWORD PTR SS:[EBP-8]
004B4216   BA C8444B00   MOV EDX,tute_.004B44C8           ASCII "Free Registry Fix can autc
004B421B   E8 2C05F5FF   CALL tute_.0040474C
004B4220   6A 31         PUSH 31
004B4222   8D55 EC       LEA EDX,DWORD PTR SS:[EBP-14]
004B4225   A1 2CC64C00   MOV EAX,DWORD PTR DS:[4CC62C]
004B422A   8B00          MOV EAX,DWORD PTR DS:[EAX]
004B422C   E8 A7A0FBFF   CALL tute_.0046E2D8
004B4231   8B45 EC       MOV EAX,DWORD PTR SS:[EBP-14]
004B4234   E8 3B09F5FF   CALL tute_.00404B74
004B4239   50            PUSH EAX
004B423A   8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]
004B423D   E8 3209F5FF   CALL tute_.00404B74
004B4242   50            PUSH EAX
004B4243   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
004B4246   E8 A90FFAFF   CALL tute_.004551F4
004B424B   50            PUSH EAX
004B424C   E8 3732F5FF   CALL <JMP.&user32.MessageBoxA>   ;Llama al mensaje..!!
004B4251   83F8 02       CMP EAX,2
004B4254 v 75 13         JNZ SHORT tute_.004B4269
004B4256   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
004B4259   8B80 18050000 MOV EAX,DWORD PTR DS:[EAX+518]
004B425F   E8 A4FBFCFF   CALL tute_.00483E08
004B4264 v E9 28020000   JMP tute_.004B4491
004B4269   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
004B426C   8B80 1C050000 MOV EAX,DWORD PTR DS:[EAX+51C]
```

Si saltara no pasaríamos por el CALL que nos muestra el mensaje, entonces… es un salto bueno ¿? Deberíamos de investigarlo, ponemos un BP en el CALL que lo controla:

```
004B41D5   E8 96710000   CALL tute_.004BB370
004B41DA   85C0          TEST EAX,EAX
004B41DC   0F84 AF020000 JE tute_.004B4491          ;ojo a este salto ¬?
004B41E2   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
004B41E5   C680 24050000 MOV BYTE PTR DS:[EAX+524],1
```

Y volvemos a pedirle que nos repare el registro, desafortunadamente debemos hacer de nuevo el Scan del mismo, bouuufff que rollo..!! después que termine le decimos ahora sí que lo repare y Olly para en el BP que hemos puesto, me llama la atención que cuando nos saca el mensaje después compara EAX con 2 si vale esto es que dijimos que no repare nada y no salta, llegando al JMP que hay detrás, este JMP apunta a :4B4491 justo donde apunta el JE que hay después del CALL en el que estamos parados, entonces como conclusión queda que este salto no nos interesa ya que si va hasta :4B4491 no realizará la función, por eso lo dejamos estar, de todos modos veamos lo que hace: hacemos un Scan y nos quedan 193 errores, entramos en el CALL en que estamos parados (en la dirección :4B41D5) y dentro hay un subcall, entramos también y vemos:

```
0041AD80   8B40 08   MOV EAX,DWORD PTR DS:[EAX+8]
0041AD83   8B40 08   MOV EAX,DWORD PTR DS:[EAX+8]
0041AD86   C3        RETN
```

Vemos que puso un valor en EAX y retorna, veamos de qué valor se trata:

```
Registers (FPU)
EAX 000000C1
ECX 00000000
EDX 00CCDD04
EBX 00CCDD04
ESP 0012FBF4
EBP 0012FC28
ESI 0044FEC8 tute_.0044FEC8
EDI 0012FD90
```

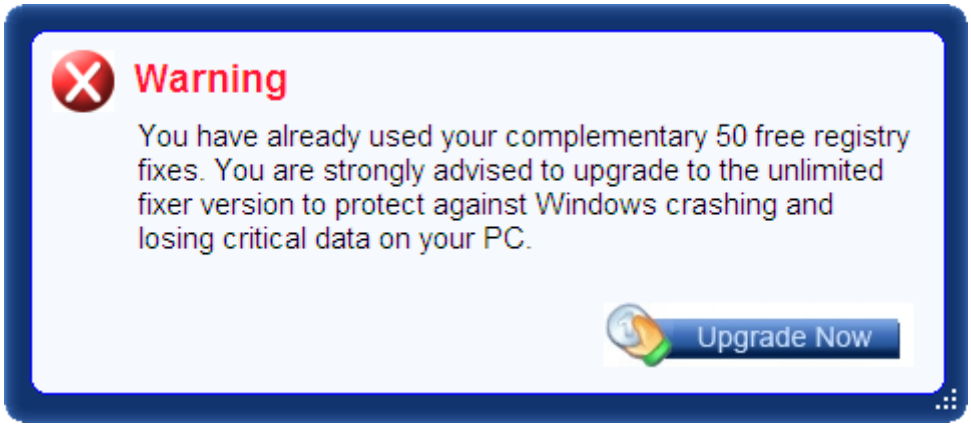
dejó EAX==0C1h, a simple vista ese valor no dice nada pero…

```
Command : 0c1        HEX: C1 - DEC: 193 - ASCII: Á
```

resulta que mira el número de errores y lo pone en EAX, claro si es 0 es que no hay error por eso a la salida del CALL nos manda a :4B4491 ya lo entendemos, quiere decir que lo que buscamos está más abajo, seguimos traceando y enseguida encontramos esta otra comparación:

```
004B41FC   E8 476AF5FF   CALL tute_.0040AC48
004B4201   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
004B4204   DC98 B0040000 FCOMP QWORD PTR DS:[EAX+4B0]
004B420A   DFE0          FSTSW AX
004B420C   9E            SAHF
004B420D   0F82 2C020000 JB tute_.004B443F
004B4213   8D45 F8       LEA EAX,DWORD PTR SS:[EBP-8]
004B4216   BA C8444B00   MOV EDX,tute_.004B44C8      ASCII "Free Registry Fix can autc
004B421B   E8 2C05F5FF   CALL tute_.0040474C
004B4220   6A 31         PUSH 31
```

Se mete en el CALL donde llama a "GetLocalTime" después hace operaciones por todos lados y al final compara en :4B4204 y hace un "FSTSW AX" que vete a saber qué hace (deberé estudiar más, jeje… y no es broma) y ya llega el salto "JB 4B443F" que si lo tomamos y vamos hacia esa dirección nos encontramos este otro regalo:



O sea que si salta significa que hemos agotado el cupo de 50 reparaciones verdad? vale no saltará nunca más, jajaa… peeeeero seguiremos viendo el mensaje cada vez y lo que es peor no reparará más de 50 entradas a la vez, necesitamos además de nopear este salto:

```
004B41FC   E8 476AF5FF    CALL tute_.0040AC48
004B4201   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
004B4204   DC98 B0040000  FCOMP QWORD PTR DS:[EAX+4B0]
004B420A   DFE0           FSTSW AX
004B420C   9E             SAHF
004B420D   90             NOP
004B420E   90             NOP
004B420F   90             NOP
004B4210   90             NOP
004B4211   90             NOP
004B4212   90             NOP
004B4213   8D45 F8        LEA EAX,DWORD PTR SS:[EBP-8]
004B4216   BA C8444B00    MOV EDX,tute_.004B44C8          ASCII "Free Registry Fix can autc
004B421B   E8 2C05F5FF    CALL tute_.0040474C
004B4220   6A 31          PUSH 31
```

Localizar donde mira el número de entradas reparadas hasta el momento además de evitar el mensaje de aviso cada vez, entonces volvemos a hacer el Scan, grrr y traceamos el CALL en el que estamos, ya sabemos para qué valen los dos primeros saltos, seguiremos investigando hasta descifrarla entera ;-)

Lo siguiente que haremos será evitar el MessageBoxA de los hue… nos situamos sobre el CALL que lo muestra y miramos la pila:

```
EBP-3C   000F091A  hOwner = 000F091A ('Free Registry Fix',class='FREEREGFIX.CLASS',parent=002006CC)
EBP-38   004B44C8  Text = "Free Registry Fix can automatically remove registry problems and this version
EBP-34   023C4C68  Title = "Free Registry Fix"
EBP-30   00000031  Style = MB_OKCANCEL|MB_ICONEXCLAMATION|MB_APPLMODAL
EBP-2C   0012FDEC  Pointer to next SEH record
EBP-28   004B44B4  SE handler
```

Son 4 parámetros que se le pasan al CALL, si lo nopeamos quedará la pila descuadrada, debemos pues cambiar el CALL por esto otro:

```
004B4243   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
004B4246   E8 A90FFAFF    CALL tute_.004551F4
004B424B   50             PUSH EAX
004B424C   83C4 10        ADD ESP,10
004B424F   B8 01000000    MOV EAX,1
004B4254 v EB 13          JMP SHORT tute_.004B4269
004B4256   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
004B4259   8B80 18050000  MOV EAX,DWORD PTR DS:[EAX+518]
004B425F   E8 A4FBFCFF    CALL tute_.00483E08
004B4264 v E9 28020000    JMP tute_.004B4491
004B4269   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
004B426C   8B80 1C050000  MOV EAX,DWORD PTR DS:[EAX+51C]
```

Además de cambiar el CALL por el "ADD ESP,10" luego veremos porqué también hemos puesto EAX==1 que es el valor de retorno cuando decimos que repare y hemos JMP'eado el salto en :4B4254 para que realice siempre la función sin advertir nada de 50 usos ni nada!!

Lo de sustituir el CALL por el "ADD ESP,10" tiene una explicación: hemos visto que se le pasaban 4 argumentos (o parámetros) que son:

[ESP]==hOwner
[ESP+4]==Text
[ESP+8]==Title
[ESP+C]==Style

Todos los parámetros de la ventana, si nopeamos el CALL si más estos quedarán almacenados en la pila y molestarán después, es por esto que debemos quitarlos de la misma, la manera es sumándole 10h al puntero de pila ESP ¿porqué 10h y no otro valor? Hagamos las cuentas: cada DWORD son 4 bytes (32 bits) si le empujamos 4 parámetros son 4 DWORDS que serían 4*4==16 bytes (en decimal ojo) pasado a hexa sería 16d==10h de ahí sale ese "ADD ESP,10" espero que se entrienda bien.

Luego el "MOV EAX,1" ya está explicado y el JMP también, de esta manera ya no vemos más el mensaje, pero no aún hará más de 50 reparaciones seguidas, vamos a por el contador, la mejor manera de pillarlo es traceando (yo aún no lo he pillado que conste) debe meterse en algún loop donde contabilice las pasadas (entradas reparadas) y la condición de salida deben ser dos:

- Que se hayan reparado todas
- Que hayamos llegado a las 50

Sabiendo esto podemos continuar a ver si hay suerte ;-) una vez saltamos hacia :4B4269 comienzan los CALL donde se supone se van a reparar las entradas o errores del registro, nosotros entraremos con F7 en todos ellos, de momento en este ya entramos:

```
004B424B   50              PUSH EAX
004B424C   83C4 10         ADD ESP,10
004B424F   B8 01000000     MOV EAX,1
004B4254 v EB 13           JMP SHORT tute_.004B4269
004B4256   8B45 FC         MOV EAX,DWORD PTR SS:[EBP-4]
004B4259   8B80 18050000   MOV EAX,DWORD PTR DS:[EAX+518]
004B425F   E8 A4FBFCFF     CALL tute_.00483E08
004B4264 v E9 28020000     JMP tute_.004B4491
004B4269   8B45 FC         MOV EAX,DWORD PTR SS:[EBP-4]
004B426C   8B80 1C050000   MOV EAX,DWORD PTR DS:[EAX+51C]
004B4272   8B40 08         MOV EAX,DWORD PTR DS:[EAX+8]
004B4275   E8 B60EFDFF     CALL tute_.00485130                ;entramos aqui..!!
004B427A   8D55 E4         LEA EDX,DWORD PTR SS:[EBP-1C]
004B427D   E8 124CF5FF     CALL tute_.00408E94
004B4282   8B4D E4         MOV ECX,DWORD PTR SS:[EBP-1C]
004B4285   8D45 E8         LEA EAX,DWORD PTR SS:[EBP-18]
004B4288   BA 1C464B00     MOV EDX,tute_.004B461C             ASCII "Problems : "
004B428D   E8 2E07F5FF     CALL tute_.004049C0
004B4292   8B55 E8         MOV EDX,DWORD PTR SS:[EBP-18]
```

Pulsamos F7 y estamos dentro, vemos un loop, jeje…

```
0048514A   8B47 04         MOV EAX,DWORD PTR DS:[EDI+4]
0048514D   8BD6            MOV EDX,ESI
0048514F   E8 345CF9FF     CALL tute_.0041AD88
00485154   E8 8BFEFFFF     CALL tute_.00484FE4
00485159   03E8            ADD EBP,EAX
0048515B   46              INC ESI
0048515C   4B              DEC EBX
0048515D ^ 75 EB           JNZ SHORT tute_.0048514A           ;fin_loop_reparador!!
0048515F   8BC5            MOV EAX,EBP
00485161   5D              POP EBP
```

En él vamos viendo como la condición de salida es cuando EBX=!0 entonces saldrá, al inicio vemos EBX==10h si somos observadores al final vemos como quedó EBP donde se va acumulando el valor de EAX a cada pasada (valor que retorna el CALL de :485154), al final vemos como EBP pasa su valor a EAX y hace un "POP EBP" entonces EAX tiene la respuesta a este loop, veámosla en la ventana de registros:

```
Registers (MMX)
EAX 000000C0
ECX 00000000
EDX 0000000F
EBX 00CCDD04
ESP 0012FBFC
EBP 0012FC28
ESI 0044FEC8 tute_.0044FEC8
EDI 0012FD90
```

Quedó EAX==0C0h que pasado a decimal supone 192 justo el núemro de errores que tengo en ete momento en mi máquina (no sé porqué en cada ejecución encuentra uno menos ¿?) en fin lo que hace es mirar el número de errores que hay, el motivo del EBX==10 es porque los errores se subdividen en categorías y de ahí que se vaya acumulando en EBP a cada pasada, lo que no sabemos de momento es si ya las ha reparado o todavía no, sigamos traceando a ver que encontramos… de momento entramos en el siguiente CALL:

```
004B427A   8D55 E4         LEA EDX,DWORD PTR SS:[EBP-1C]
004B427D   E8 124CF5FF     CALL tute_.00408E94                ;Ahora entramos aqui..!!
004B4282   8B4D E4         MOV ECX,DWORD PTR SS:[EBP-1C]
004B4285   8D45 E8         LEA EAX,DWORD PTR SS:[EBP-18]
004B4288   BA 1C464B00     MOV EDX,tute_.004B461C             ASCII "Problems : "
004B428D   E8 2E07F5FF     CALL tute_.00404 9C0
004B4292   8B55 E8         MOV EDX,DWORD PTR SS:[EBP-18]
```

Me meto con F7 como digo y comienzo a tracear atento a los registros, de repente me veo en un bucle donde se va almacenando un valor, el loop es este:

```
00408E64   56              PUSH ESI
00408E65   31D2            XOR EDX,EDX
00408E67   F7F1            DIV ECX
00408E69   4E              DEC ESI
00408E6A   80C2 30         ADD DL,30
00408E6D   80FA 3A         CMP DL,3A
00408E70 v 72 03           JB SHORT tute_.00408E75
00408E72   80C2 07         ADD DL,7
00408E75   8816            MOV BYTE PTR DS:[ESI],DL
00408E77   09C0            OR EAX,EAX
00408E79 ^ 75 EA           JNZ SHORT tute_.00408E65
00408E7B   59              POP ECX
00408E7C   5A              POP EDX
```

Cuando termina miro a ver qué es lo que almacenó:

```
Address   Hex dump                                          ASCII
0012FBF1  31 39 32 C8 FE 44 00 82 42 4B 00 EC FD 12 00 B4  192╚■D.éBK.ÿ²♯.┤
0012FC01  44 4B 00 28 FC 12 00 04 DD CC 00 00 00 00 00 00  DK.(ᵃ♯.♦¦╠......
0012FC11  00 00 00 68 4C 3C 02 00 00 00 00 00 00 00 00 C8  ...hL<☻........╚
0012FC21  44 4B 00 7C 24 CC 00 84 FD 12 00 D5 3B 4B 00 32  DK.|$╠.ä²♯.'':K.2
```

O sea que almacena el número de errores en ese momento, después copia este 192 a otra zona creada por el progama en tiempo de ejecución, esto lo hace en este CALL:

```
004047BE   89F0          MOV EAX,ESI
004047C0   E8 33E5FFFF   CALL tute_.00402CF8
004047C5   89D8          MOV EAX,EBX
004047C7   E8 E8FEFFFF   CALL tute_.004046B4                ;hace las operaciones...
004047CC   893B          MOV DWORD PTR DS:[EBX],EDI         ;guarda el resultado!!
004047CE   5F            POP EDI                            0012FD90
004047CF   5E            POP ESI
004047D0   5B            POP EBX
004047D1   C3            RETN
```

Dentro hace lo explicado y retorna un valor en EDI que se guarda ¿? Veamos que es:

```
Registers (3DNow!)
EAX 0012FC0C
ECX 00000000
EDX 00000000
EBX 0012FC0C
ESP 0012FBD4
EBP 0012FC28
ESI 0012FBF1
EDI 00CF4B3C ASCII "192"
```

jeje.. nuestro 192, vaya vaya nos vamos acercando, veamos que hace con él, salimos del CALL en el que habíamos entrado, llegamos a esta parte:

```
004B427A   8D55 E4       LEA EDX,DWORD PTR SS:[EBP-1C]
004B427D   E8 124CF5FF   CALL tute_.00408E94                ;Ahora entramos aqui..!!
004B4282   8B4D E4       MOV ECX,DWORD PTR SS:[EBP-1C]
004B4285   8D45 E8       LEA EAX,DWORD PTR SS:[EBP-18]
004B4288   BA 1C464B00   MOV EDX,tute_.004B461C             ASCII "Problems : "
004B428D   E8 2E07F5FF   CALL tute_.004049C0
004B4292   8B55 E8       MOV EDX,DWORD PTR SS:[EBP-18]
```

Si miramos los registros lo entenderemos todo mucho mejor:

```
Registers (3DNow!)
EAX 0012FC10
ECX 00CF4B3C ASCII "192"
EDX 004B461C ASCII "Problems : "
EBX 00CCDD04
ESP 0012FBFC
```

Parece que se está preparando para mostrarnos el número de problemas encontrados, si seguimos traceando entrando en los CALL con F7 al final encontramos una zona muy interesante, es esta:

```
0049254B   55            PUSH EBP
0049254C   B8 03000000   MOV EAX,3
00492551   E8 56FCFFFF   CALL tute_.004921AC                ;repara algunos errores..!!
00492556   59            POP ECX
00492557   50            PUSH EAX
00492558   8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]       ;[EBP-8]==50 => contador?
0049255B   5A            POP EDX
0049255C   2BC2          SUB EAX,EDX
0049255E   85C0          TEST EAX,EAX
00492560 v 7E 16         JLE SHORT tute_.00492578
00492562   55            PUSH EBP
00492563   B8 02000000   MOV EAX,2
00492568   E8 3FFCFFFF   CALL tute_.004921AC
0049256D   59            POP ECX
0049256E   50            PUSH EAX
0049256F   8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]       ;queda EAX==32
00492572   2B45 F4       SUB EAX,DWORD PTR SS:[EBP-C]       ;le resta otros 32!
00492575   5A            POP EDX
00492576   2BC2          SUB EAX,EDX
00492578   85C0          TEST EAX,EAX                       ;es EAX==0
0049257A v 7E 16         JLE SHORT tute_.00492592           ;y salta..!!
0049257C   55            PUSH EBP
0049257D   B8 01000000   MOV EAX,1
00492582   E8 25FCFFFF   CALL tute_.004921AC
00492587   59            POP ECX
00492588   50            PUSH EAX
00492589   8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]
0049258C   2B45 F4       SUB EAX,DWORD PTR SS:[EBP-C]
0049258F   5A            POP EDX
00492590   2BC2          SUB EAX,EDX
00492592   85C0          TEST EAX,EAX
00492594 v 7E 09         JLE SHORT tute_.0049259F
00492596   55            PUSH EBP
```

Esta parte parece interesante, juega mucho con [EBP-8] que contiene el valor 32h que pasado a decimal queda 50d justo la limitación..!! si miramos al inicio de la rutina encontramos aquí su inicialización a 32h:

```
00492482   53            PUSH EBX
00492483   33DB          XOR EBX,EBX
00492485   895D E8       MOV DWORD PTR SS:[EBP-18],EBX
00492488   895D E4       MOV DWORD PTR SS:[EBP-1C],EBX
0049248B   894D F0       MOV DWORD PTR SS:[EBP-10],ECX
0049248E   8955 F8       MOV DWORD PTR SS:[EBP-8],EDX       ;guarda un 50!!
00492491   8945 FC       MOV DWORD PTR SS:[EBP-4],EAX
00492494   8B5D 08       MOV EBX,DWORD PTR SS:[EBP+8]
```

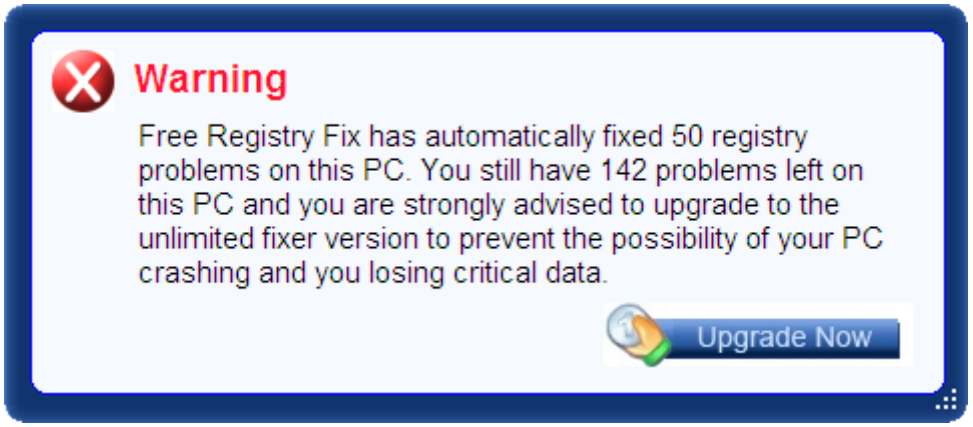
Pero no nos alarmemos, primero estudiemos esto a ver si conseguimos sacar algo en claro, lo que es evidente que los "CALL tute_.004921AC" son reparadores de entradas, y que coge el valor de [EBP-8] le va restando hasta que al final comienza a saltar evitando

estos CALL, yo juraría que está evitando reparar más de 50 y lo controla a través de la susodicha variable local [EBP-8].

Un primer parche podría ser pasarle un valor elevado en la inicialización del flag, ver de donde toma EDX su valor y forzar a que tome uno mucho más alto, también valdría la pena probar a nopear los saltos que evitan pasar por los CALL reparadores a ver si así igual funciona, debemos decir antes de nada que estamos dentro de este CALL:

```
004B4360   6A 00          PUSH 0
004B4362   33C9           XOR ECX,ECX
004B4364   BA 32000000    MOV EDX,32
004B4369   E8 0EE1FDFF    CALL tute_.0049247C                    ;Aqui lo hace todo..!!
004B436E   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
004B4371   8B80 B8040000  MOV EAX,DWORD PTR DS:[EAX+4B8]
```

Y es aquí donde se hace lo último que hemos comentado, o sea que en este CALL está la restricción de 50 reparaciones por ejecución, recordemos que podemos reparar cuanto queramos pero de 50 en 50 ya que hemos nopeado el salto que hace que el programa expire y no reapare más (que estaba en :4B420D) a la salida del CALL pulsamos F9 y sale esto:



O sea que reparó los 50 de rigor y nos dejó 142 para otro día no? Jeje.. Bueno seguimos que queda bien poco, lo primero que haremos será investigar de dónde obtiene EDX su valor antes de pasarlo a [EBP-8] y cambiarlo por EDX==0000C350 que son 50.000 decimal, jeje.. sólo quiere reparar 50 entradas? Pues ahora va a reparar cincuentamil ;-) eso intentaremos, de momento vuelta a escanear, cuando para nos detenemos en el CALL que hace todo que está en :4B4369 y entramos con F7, pero que demonios, mirad..!!

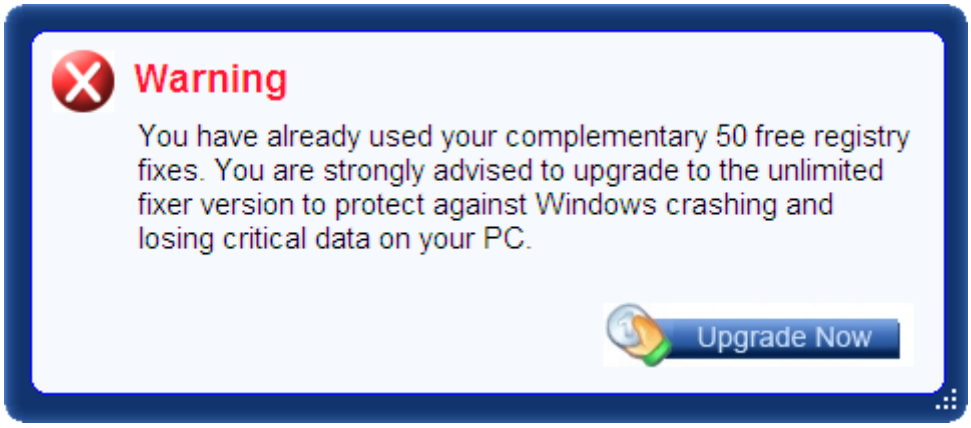
```
004B4340   8B80 B8040000  MOV EAX,DWORD PTR DS:[EAX+4B8]
004B4346   8B55 FC        MOV EDX,DWORD PTR SS:[EBP-4]
004B4349   8950 0C        MOV DWORD PTR DS:[EAX+C],EDX
004B434C   C740 08 7C474B MOV DWORD PTR DS:[EAX+8],tute_.004B477C
004B4353   8B55 FC        MOV EDX,DWORD PTR SS:[EBP-4]
004B4356   8950 2C        MOV DWORD PTR DS:[EAX+2C],EDX
004B4359   C740 28 D0464B MOV DWORD PTR DS:[EAX+28],tute_.004B46D
004B4360   6A 00          PUSH 0
004B4362   33C9           XOR ECX,ECX
004B4364   BA 32000000    MOV EDX,32                             ;Aqui la limitacion..!!
004B4369   E8 0EE1FDFF    CALL tute_.0049247C                    ;Aqui lo hace todo..!!
004B436E   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
004B4371   8B80 B8040000  MOV EAX,DWORD PTR DS:[EAX+4B8]
```

hay un “MOV EX,32” justo antes del CALL, jejeje… cambiémoslo por esto otro a ver:

```
004B4340   8B80 B8040000  MOV EAX,DWORD PTR DS:[EAX+4B8]
004B4346   8B55 FC        MOV EDX,DWORD PTR SS:[EBP-4]
004B4349   8950 0C        MOV DWORD PTR DS:[EAX+C],EDX
004B434C   C740 08 7C474B MOV DWORD PTR DS:[EAX+8],tute_.004B477C
004B4353   8B55 FC        MOV EDX,DWORD PTR SS:[EBP-4]
004B4356   8950 2C        MOV DWORD PTR DS:[EAX+2C],EDX
004B4359   C740 28 D0464B MOV DWORD PTR DS:[EAX+28],tute_.004B46D
004B4360   6A 00          PUSH 0
004B4362   33C9           XOR ECX,ECX
004B4364   BA 50C30000    MOV EDX,0C350                          ;Elevamos el limite ;-)
004B4369   E8 0EE1FDFF    CALL tute_.0049247C                    ;Aqui lo hace todo..!!
004B436E   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
004B4371   8B80 B8040000  MOV EAX,DWORD PTR DS:[EAX+4B8]
```

Y damos RUN directamente, vemos que se está cargando los bloques que da gusto, jeje.. lo repara TODO, pero queeeeeeee…

El muy cabr… nos vuelve a pillar cuando sólo quedaban 11 errores por reparar:



Está claro que nos estaban esperando eh! Hay chequeos por todas partes, a ver este de donde diantres sale ¿?

Si entramos de nuevo en la ruitina encontramos el contador de errores reparados:

```
0049256E   50            PUSH EAX
0049256F   8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]      ;queda EAX==32
00492572   2B45 F4       SUB EAX,DWORD PTR SS:[EBP-C]      ;le resta otros 32!
00492575   5A            POP EDX
00492576   2BC2          SUB EAX,EDX
00492578   85C0          TEST EAX,EAX                      ;es EAX==0
0049257A   7E 16         JLE SHORT tute_.00492592          ;y salta..!!
0049257C   55            PUSH EBP
0049257D   B8 01000000   MOV EAX,1
00492582   E8 25FCFFFF   CALL tute_.004921AC
00492587   59            POP ECX
```

El contador lo tiene en [EBP-C], en esta última ejecución quedaron 10 errores y tras superar algunos CALL iba viendo como se actualizaba, ahora mismo lleva 8:

```
Address   Hex dump                                          ASCII
0012FBD8  08 00 00 00 50 C3 00 00 74 91 D7 00 28 FC 12 00  ■...PÃ..t‘×.(ü↕.
0012FBE8  6E 43 4B 00 00 00 00 00 FC FB 12 00 91 43 4B 00  nCK.....ü¹↕.‘CK.
0012FBF8  28 FC 12 00 EC FD 12 00 B4 44 4B 00 28 FC 12 00  (ü↕.ì²↕.´DK.(ü↕.
```

Esos 8 indica que ha reparado este número de errores, quizá esto sea lo que verifique más adelante para mostrarnos el mensaje que antes vimos, de momento seguimos traceando y nos topamos mira por donde con el CALL que saca el último mensaje, veamos la rutina:

```
004B63B4   53            PUSH EBX                          ;rutina_msg_final
004B63B5   56            PUSH ESI
004B63B6   57            PUSH EDI
004B63B7   51            PUSH ECX
004B63B8   8BF2          MOV ESI,EDX
004B63BA   8BD8          MOV EBX,EAX
004B63BC   8BCB          MOV ECX,EBX
004B63BE   B2 01         MOV DL,1
004B63C0   A1 10C14A00   MOV EAX,DWORD PTR DS:[4AC110]
004B63C5   E8 860AFBFF   CALL tute_.00466E50               ;llama a MsgBeep..!!
004B63CA   890424        MOV DWORD PTR SS:[ESP],EAX
004B63CD   8B7E 04       MOV EDI,DWORD PTR DS:[ESI+4]
004B63D0   8B76 08       MOV ESI,DWORD PTR DS:[ESI+8]
004B63D3   C683 24050000 MOV BYTE PTR DS:[EBX+524],0
004B63DA   8BC3          MOV EAX,EBX
004B63DC   E8 17C7FFFF   CALL tute_.004B2AF8
004B63E1   80BB C1040000 CMP BYTE PTR DS:[EBX+4C1],0       ;mira si quedan errores aun..!!
004B63E8   75 0E         JNZ SHORT tute_.004B63F8
004B63EA   8BCE          MOV ECX,ESI
004B63EC   8BD7          MOV EDX,EDI
004B63EE   8B0424        MOV EAX,DWORD PTR SS:[ESP]
004B63F1   E8 2A61FFFF   CALL tute_.004AC520
004B63F6   EB 08         JMP SHORT tute_.004B6400
004B63F8   8B0424        MOV EAX,DWORD PTR SS:[ESP]
004B63FB   E8 0C64FFFF   CALL tute_.004AC80C
004B6400   8B0424        MOV EAX,DWORD PTR SS:[ESP]
004B6403   8B10          MOV EDX,DWORD PTR DS:[EAX]
004B6405   FF92 EC000000 CALL DWORD PTR DS:[EDX+EC]        ;Aqui el mensaje..!!
004B640B   8BF0          MOV ESI,EAX
```

Es parte de la misma, llama a MessageBeep en :4B63C5, después hace una interesante comparación en :4B63E1 si salta en el JNZ posterior nos saca el mensaje mangui y si no lo hace nos saca uno parecido pero diciendo que nos quedan errores en el sistema, de momento eliminaremos el segundo de ellos y el MessageBeep que agobia bastante (al menos a mi) para el ruidito entramos en la rutina hasta encontrar esto:

```
0040D39C   6A 00         PUSH 0
0040D39E   E8 DDA0FFFF   CALL <JMP.&user32.MessageBeep>
0040D3A3   C3            RETN
```

La famosa llamada, la nopeamos para que no moleste más, jajaja… la dejamos así:

```
0040D39C   90          NOP
0040D39D   90          NOP
0040D39E   90          NOP
0040D39F   90          NOP
0040D3A0   90          NOP
0040D3A1   90          NOP
0040D3A2   90          NOP
0040D3A3   C3          RETN
```

Y ya tenemos un problema menos ;-) Para no ver el segundo mensaje agobiante hacemos lo propio pero ahora en esta otra zona:

```
004B6403   8B10        MOV EDX,DWORD PTR DS:[EAX]
004B6405   B8 01000000 MOV EAX,1                  ;Aqui el mensaje..!!
004B640A   90          NOP
004B640B   8BF0        MOV ESI,EAX
004B640D   833C24 00   CMP DWORD PTR SS:[ESP],0
004B6411   74 07       JE SHORT tute_.004B641A
```

Y ya no aparecerá más, ahora salvamos todos los cambios a un único archivo llamado por ejemplo tute1.exe y podemos utilizarlo tanto como queramos, he de reconocer que no termina de gustarme como queda porque al final siempre deja una entrada por reparar pero bueno y tampoco cierra la pantallita pero bien repara bastante diría yo, lejos de las 50 que nos daba, jeje..

Un placer escribir el tute de verdad, pronto estaremos com más protecciones que estudiar que afortunadamente para nosotros hay bastantes ;-) un saludo como no para todo CracksLatinoS de los que aprendo a diario. Hasta la próxima…

SyXe'05 … [hAcK-c0d3d] … [CracksLatinoS]

Para cualquier sugerencia, pregunta, duda o crítica ya sabéis donde encontrarme verdad?

syxe00@yahoo.es